

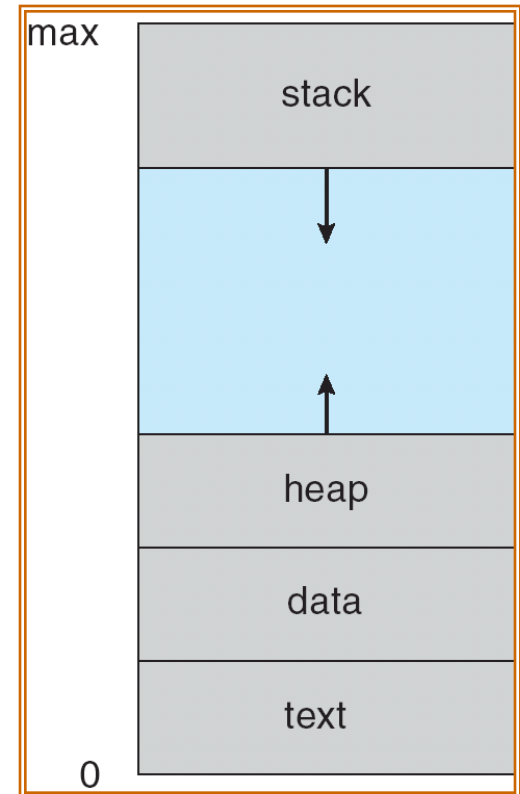
CMPS 405: OPERATING SYSTEMS

Processes and System Calls

This material is based on the operating system books by Silberschatz and Stallings

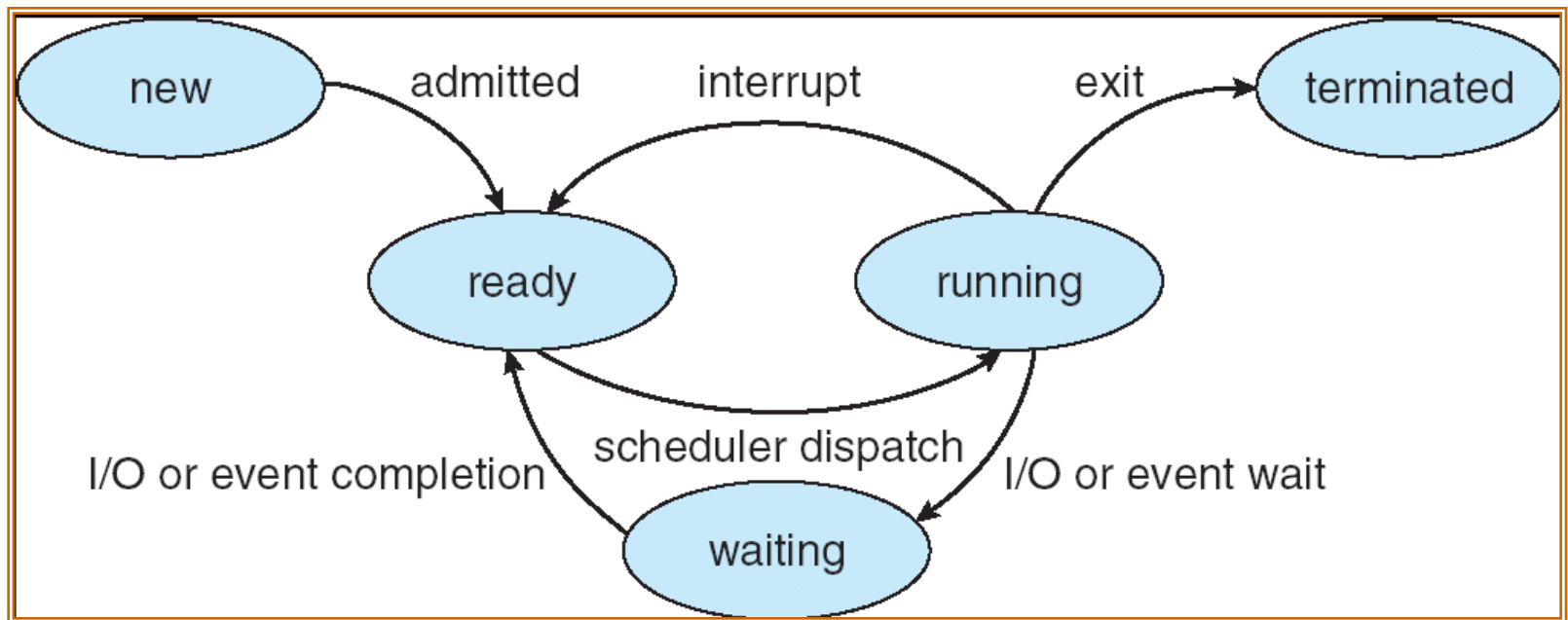
Process Concept

- **Process** – a program in execution; process execution must progress in sequential fashion
- A **process** is represented by:
 - **Text section (program code)**
 - **Current activity** represented by the program counter and the contents of the CPU registers
 - **Stack** containing **temporary data** such as **local variables** and **methods parameters**.
 - **Data section** containing **global variables**
 - **Heap** for **dynamic** memory allocation



Process State

- ❖ As a **process executes**, it changes *state*
 - **new**: The process is being created
 - **ready**: The process is waiting to be assigned to a processor
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **terminated**: The process has finished execution



Processes in Unix

- ❖ UNIX OS supports two types of process: **System Processes** and **User Processes**

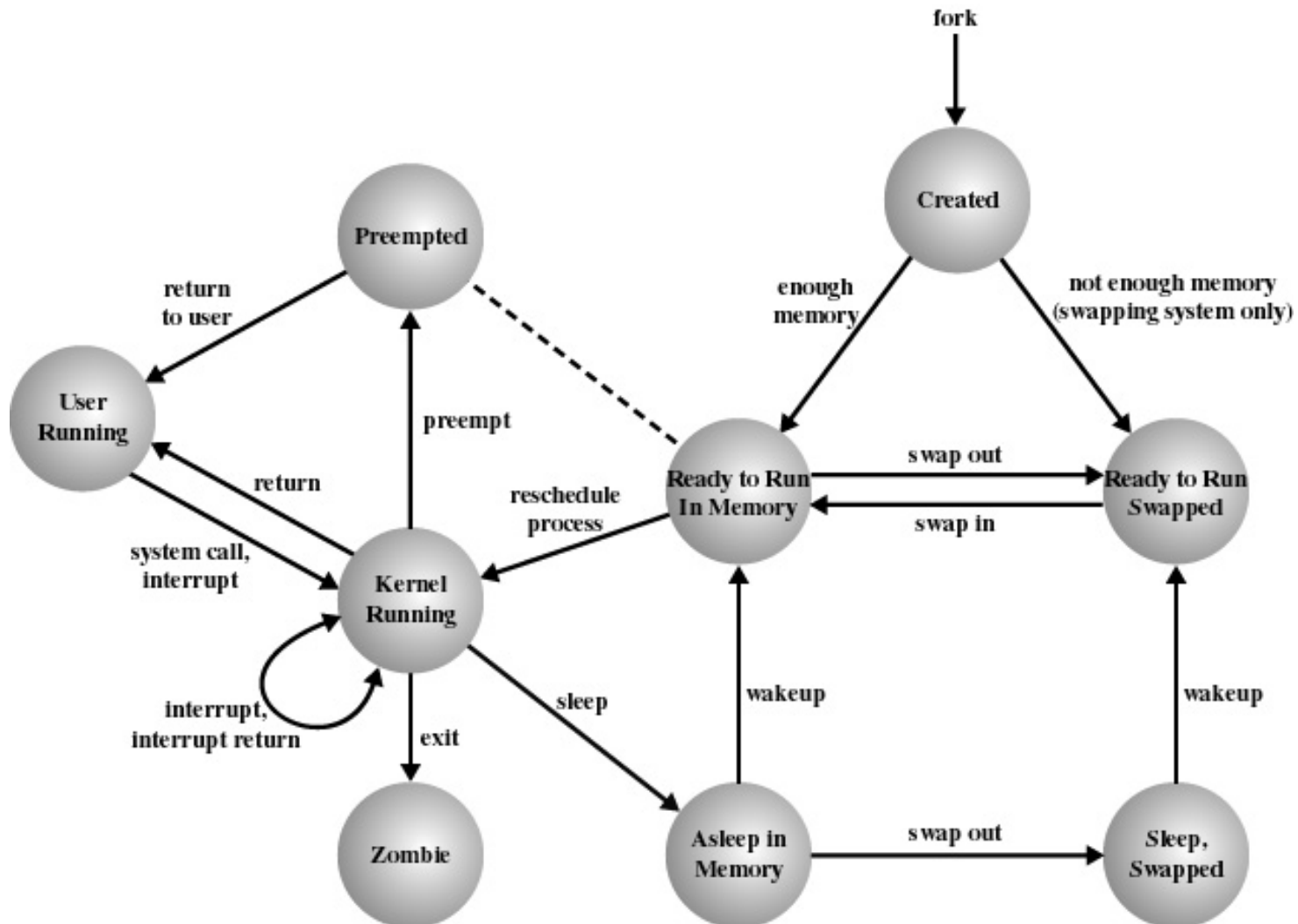
- ❖ **System Processes** run in **Kernel mode**
 - Executes operating system code to perform administrative and housekeeping functions

- ❖ **User Processes**
 - Operate in **user mode** to execute user programs and utilities
 - Operate in **kernel mode** to execute **instructions** that belong to the kernel
 - Enter **kernel mode** by issuing a **system call**, when an **exception is generated**, or when an **interrupt occurs**

Process states in UNIX

User Running	Executing in user mode.
Kernel Running	Executing in kernel mode.
Ready to Run, in Memory	Ready to run as soon as the kernel schedules it.
Asleep in Memory	Unable to execute until an event occurs; process is in main memory (a blocked state).
Ready to Run, Swapped	Process is ready to run, but the swapper must swap the process into main memory before the kernel can schedule it to execute.
Sleeping, Swapped	The process is awaiting an event and has been swapped to secondary storage (a blocked state).
Preempted	Process is returning from kernel to user mode, but the kernel preempts it and does a process switch to schedule another process.
Created	Process is newly created and not yet ready to run.
Zombie	Process no longer exists, but it leaves a record for its parent process to collect.

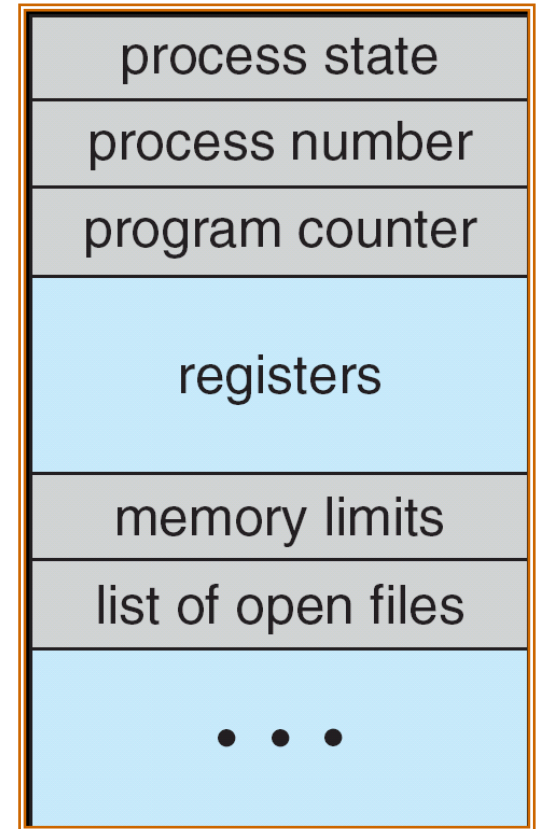
Process states in UNIX: Transition Diagram



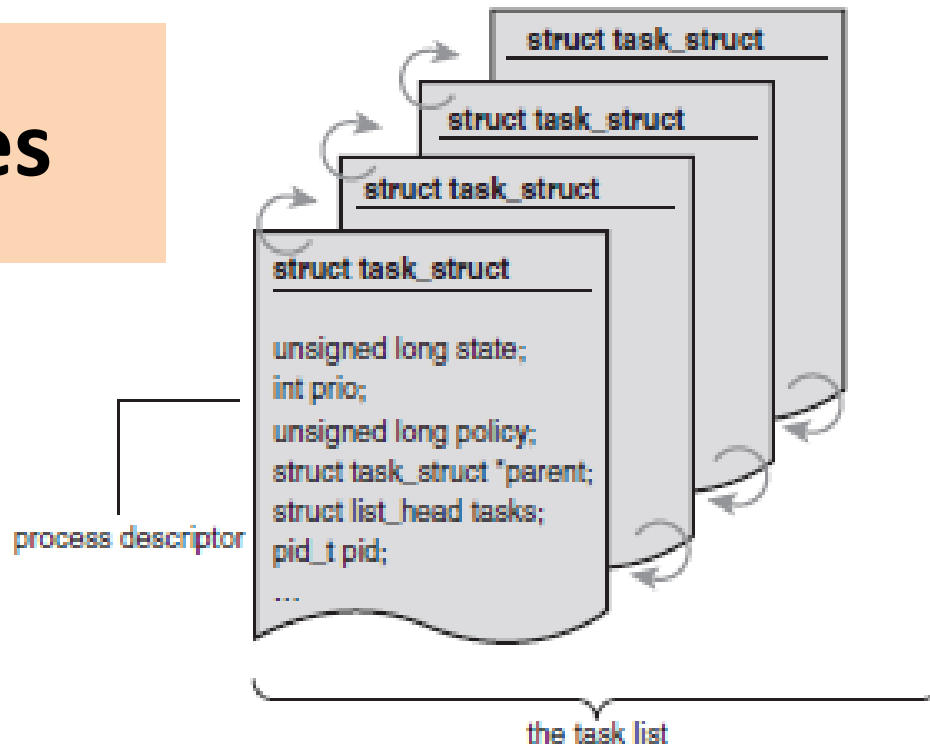
Process Control Block (PCB)

❖ A **data structure** containing **information** associated with **each process**

- Process state
- Process number
- Program counter
- CPU registers
- CPU scheduling information
- Memory-management information
- Accounting information
- List of open files
- I/O status information

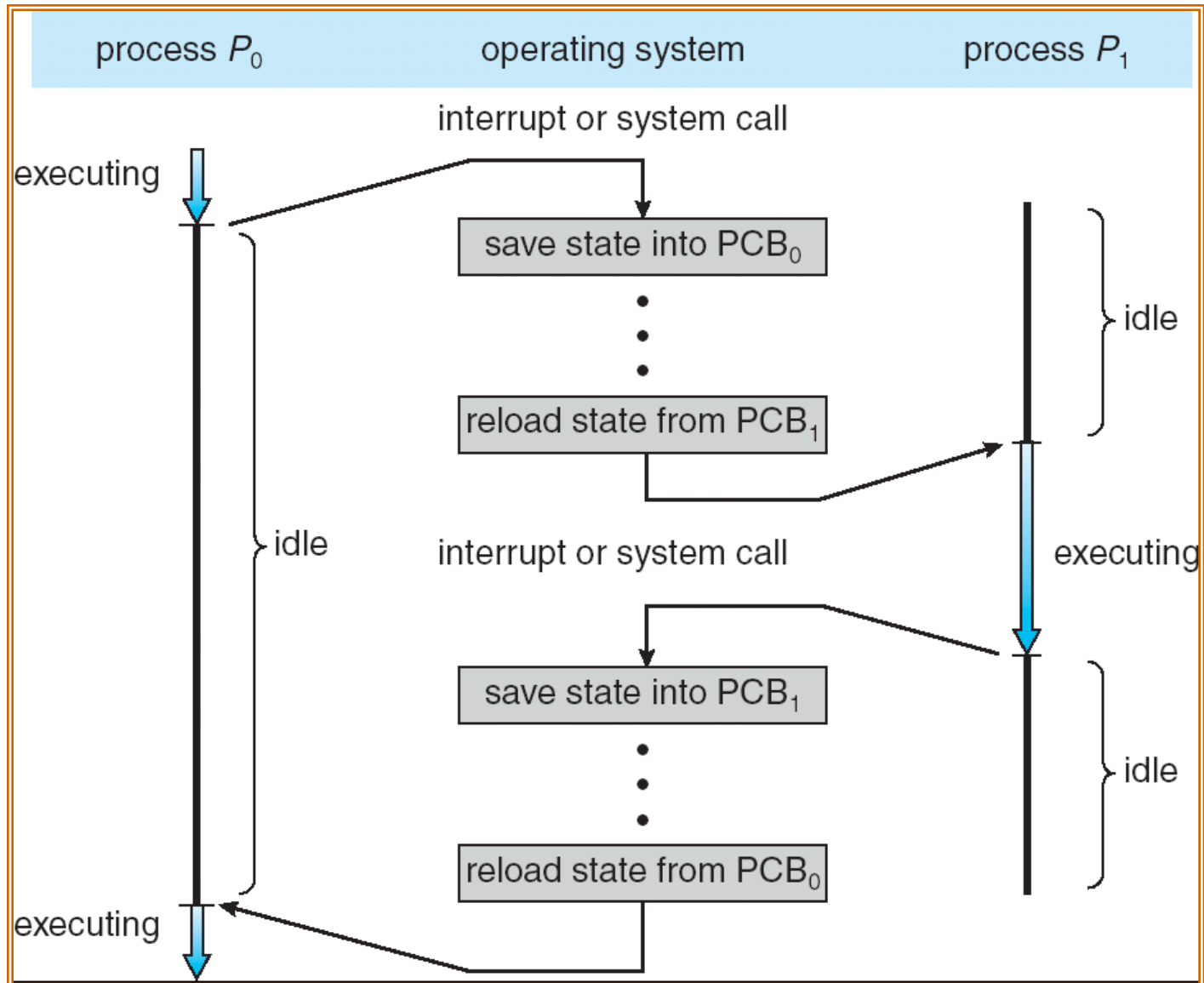


Processes



- ❖ The **kernel** stores the list of processes information in a circular doubly linked list called the ***task list***.
- ❖ Each element in the **task list** is a **process descriptor** of the type `struct task_struct`, which is defined in `<linux/sched.h>`.
- ❖ The **process descriptor** contains all the information about a specific **process** (1.7 kilobytes in size).

CPU Switching From Process to Process



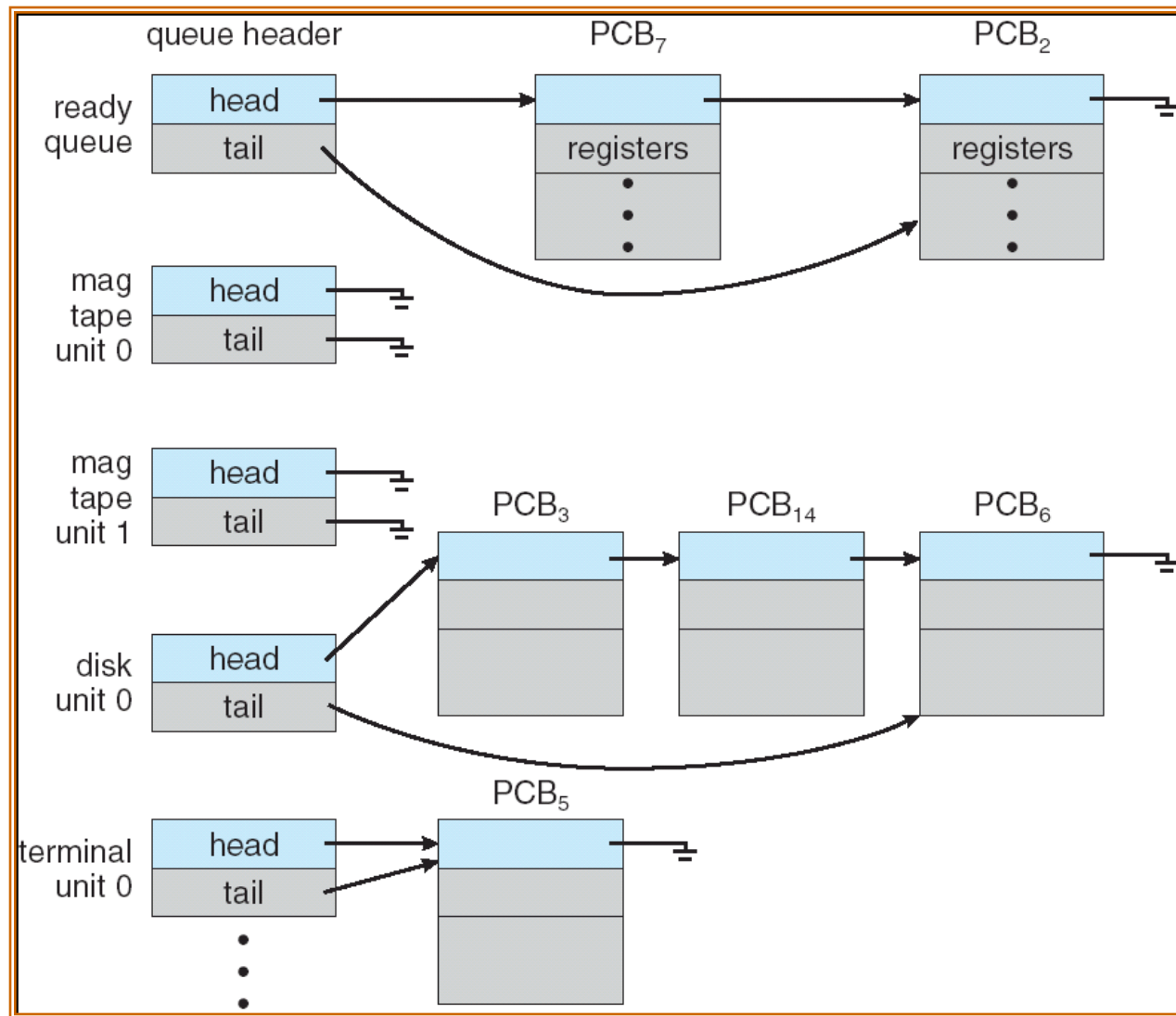
Context Switch

- ❖ When **CPU switches to another process**, the system must **save the state of the old process** and **load the saved state** for the **new process**
- ❖ **Context-switch time is overhead**; the system does no useful work while switching
- ❖ **Time dependent on hardware support**
 - Memory speed
 - Number of registers to be copied
 - The existence of special instructions (such as one instruction to load/store all registers)
 - Hardware support: UltraSPARC provide multiple sets of registers.

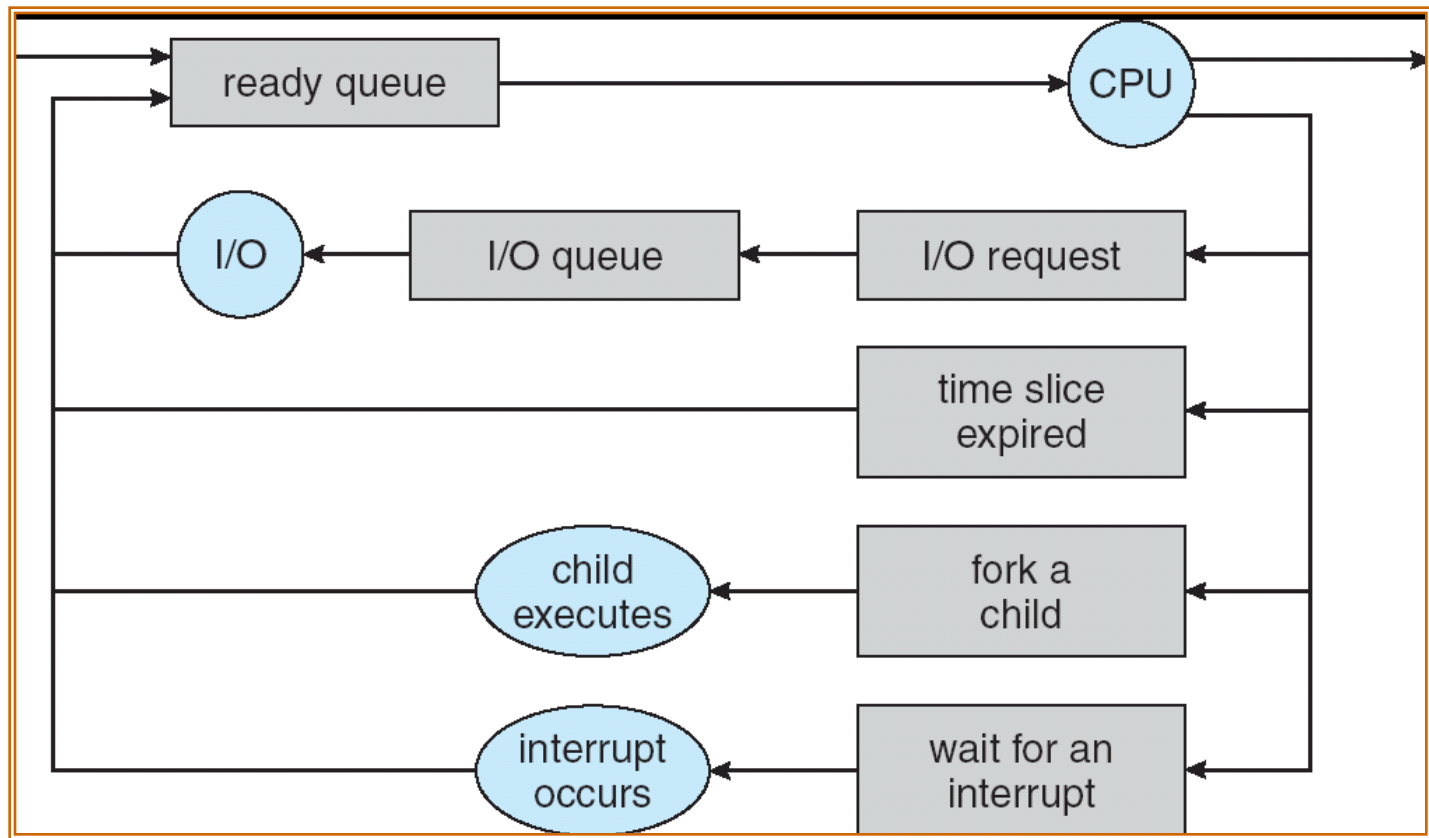
Process Scheduling Queues

- ❖ **Job queue** – set of all processes in the system
- ❖ **Ready queue** – set of all processes residing in **main memory**, **ready** and **waiting** to execute
- ❖ **Device queues** – set of processes **waiting** for an **I/O device**
- ❖ Processes migrate among the various queues

Ready Queue And Various I/O Device Queues



Representation of Process Scheduling



Schedulers

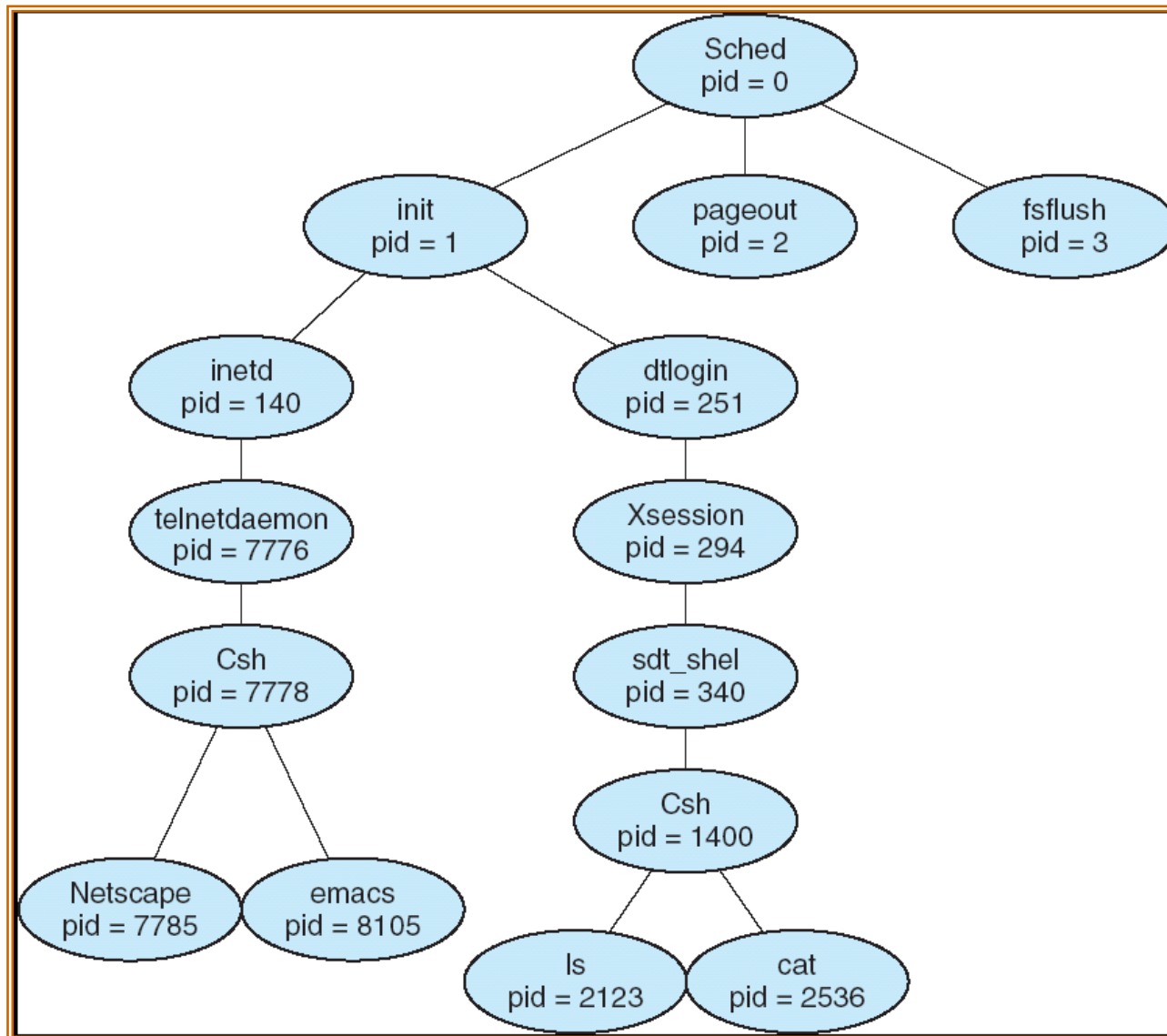
- ❖ **Long-term scheduler** (or **job scheduler**) – selects which **processes** should be **brought into the ready queue**
- ❖ **Short-term scheduler** (or **CPU scheduler**) – selects which **process** should be **executed next** and **allocates CPU**

Schedulers

- ❖ **Short-term scheduler** is invoked very **frequently (milliseconds)**
⇒ (must be fast)
- ❖ **Long-term scheduler** is invoked very **infrequently (seconds, minutes)** ⇒ (may be slow)
- ❖ The long-term scheduler controls the *degree of multiprogramming* (number of processes in memory)
- ❖ **Processes** can be **described** as either:
 - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
 - **CPU-bound process** – spends more time doing computations; few very long CPU bursts

CPU Burst: In CPU scheduling, a CPU burst refers to the period of time during which a process executes on the CPU without interruption.

A tree of processes on a typical Solaris

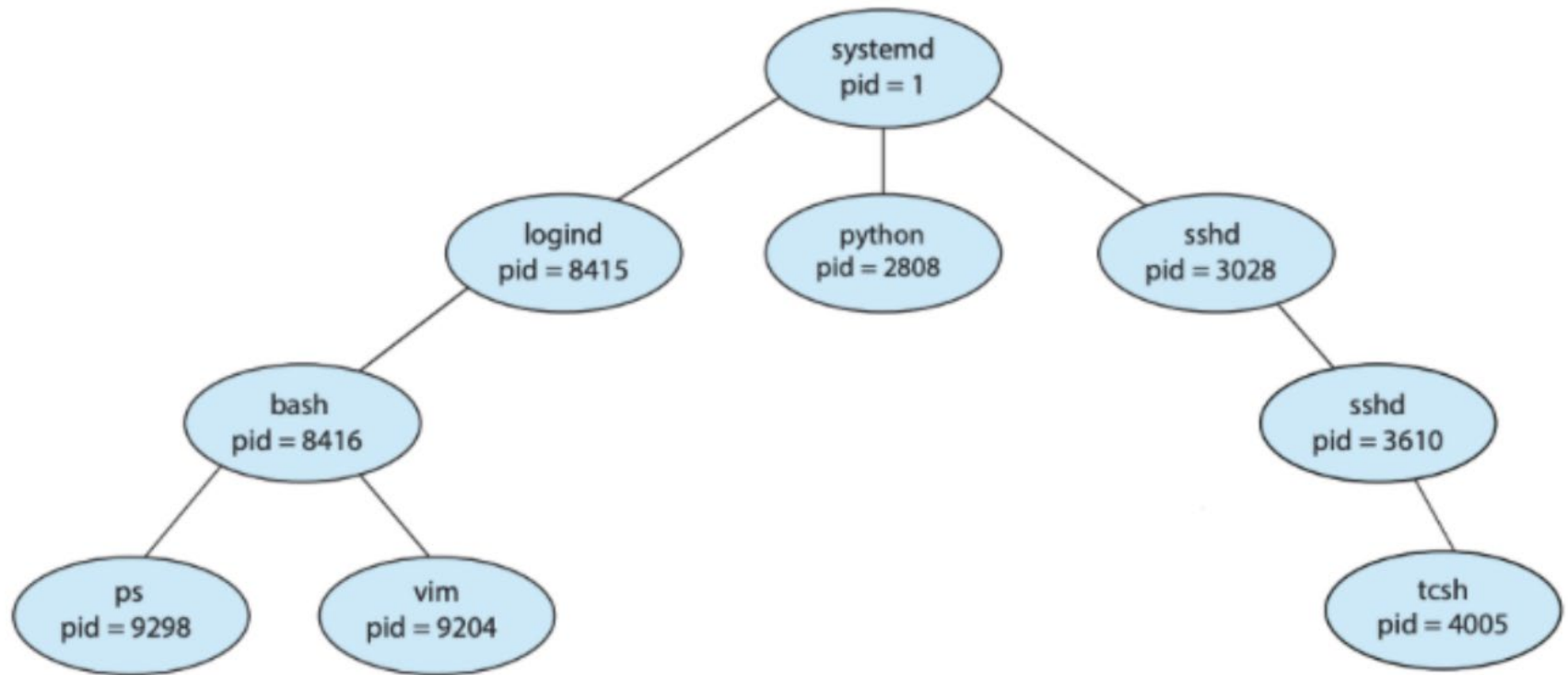




A tree of processes on a typical Solaris

- ❖ **sched**: Kernel process responsible for scheduling CPU time among processes.
- ❖ **init**: The first process started during boot, responsible for initializing and managing system services.
- ❖ **pageout**: Handles memory management by moving inactive pages from RAM to swap space.
- ❖ **inetd**: A super-server daemon managing network services.
- ❖ **dtlogin**: A graphical login manager for Unix desktop environments

A tree of processes on a typical Linux



A tree of processes on a typical **Linux**

- ❖ **systemd**: Manages system boot, services, and overall system state in Linux.
- ❖ **logind**: Manages user logins, sessions, and power-related tasks.
- ❖ **sshd**: Handles secure remote access via SSH (secure shell).



Viewing processes in Linux

❖ Can **view running processes** by various options:

1. Any of the commands:
 - `ps aux`
 - `pstree`
 - `top`
 - `htop`
2. System > Administration > System Monitor

System calls

- ❖ System calls allow user-level processes to request services of the operating system.
- ❖ Generally available as **assembly-language instructions**.
- ❖ **Some Languages** defined to **replace assembly language** for systems programming allow system calls to be made directly (e.g., C, C++).
 - **high-level languages** support **system calls** by **hidden run-time support system** (the set of **functions built into libraries** with a **compiler**) provided by the operating system.
- ❖ System programs can be thought of as **bundles of useful system calls**. They provide **basic functionality** to users and so users do not need to write their own programs to solve common problems.

System calls parameters

- ❖ **System call parameters are passed to the operating system** in three ways:
 - **Pass parameters to registers.** Only works when number of parameters is less or equal to number of available registers.
 - **Parameters are stored in a block or table in memory** then the address of that block is passed to a register. (Linux)
 - **Parameters are pushed onto the stack and popped by the operating system.**

Categories of system calls

❖ **Process control:**

- create, load, execute, abort, wait, get and set attributes and terminate.

❖ **File management:**

- create, open, read, write, get and set attributes, close and delete.

❖ **Device management:**

- request, read, write, get and set attributes and release.

❖ **Communications:**

- create, send, receive, status and delete.

❖ **Information maintenance:**

- get and set time and date, get and set system info, get and set process, file and device info.

Sample of System calls

GENERAL CLASS	SPECIFIC CLASS	SYSTEM CALL
Process Related	Process Creation & Termination	fork()
		exec()
		wait()
		exit()
	Process Identity	getpid()
		getppid()
	Process Control	signal()
		kill()
		alarm()
Interprocess Communication	Pipelines	pipe()
	Messages	msgget()
		msgsnd()
		msgrcv()
		msgctl()
	Semaphores	semget()
		semop()
	Shared Memory	shmget()
		shmat()
		shmdt()

Process Creation

- ❖ **Parent process create children processes**, which, in turn create other processes, forming a **tree of processes**.
 - **Resource sharing:** Parent and children might share all resources, subset of parent's resources, or no resources.
 - **Execution:**
 - Parent and children execute concurrently, or
 - Can have the parent to wait until children terminate.
 - **Address space:**
 - Child process duplicate of parent process (it has same program and data as the parent), or
 - A child has a new program loaded into it.
- ❖ **UNIX/Linux examples:** **fork** system call creates new process while an **exec** system call used after a **fork** to replace the process' memory space with a new program.

fork() system call

In **UNIX** the **init** program is the root of all of your processes.

- ❖ UNIX examples: **fork** system call creates new process which is a duplicate copy of the parent.
 - The result of successful **fork()** is two processes: the original process called the **parent** and a new process called the **child**.
 - **fork()** returns an **integer value** to each of these processes as follows:
 - a **positive** integer returned to the parent process which represents the child's ID.
 - **Zero** returned to the child process
 - If **fork()** fails it returns a **negative** integer returned to the original process (in this case there is only one process no new process was created)

Typically fork() Calling Sequence

```
if ( ( pid = fork () ) < 0 )  
  
    printf ("error forking\n");  
  
else if ( pid == 0 ) {  
  
    /* execute child stuff here */  
  
}  
else {  
  
    /* place parent stuff here */  
  
}
```

Current Process Information

❖ To get **process identification numbers**:

➤ **pid_t getpid(void);**

- gets the current process id

➤ **pid_t getppid(void);**

- gets the parent's process id of the current process

❖ **pid_t** is defined in the **unistd.h** and **sys/types.h** header files.
The default **max pid** is **32,768**.

fork() Example

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

void main(){
    pid_t x;
    x = fork();
    if(x<0)
        printf("fork failed.");
    else if(x>0){
        wait(NULL);
        printf("I am the parent[PID=%d].\nMy parent[PID=%d].\n",getpid(),getppid());
    }
    else{
        printf("I am the child[PID=%d].\nMy parent[PID=%d].\n",getpid(),getppid());

    }
    printf("Things that are to be done by both parent and child [PID=%d].\n",getpid());
}
```

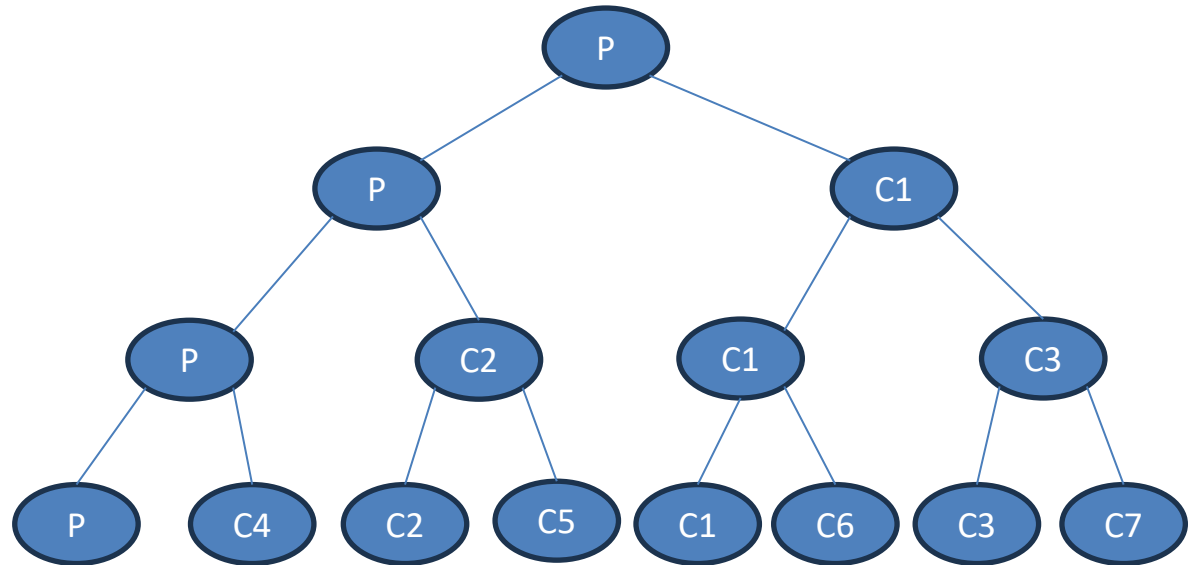
```
I am the child[PID=2735].
My parent[PID=2734].
Things that are to be done by both parent and child [PID=2735].
I am the parent[PID=2734].
My parent[PID=2642].
Things that are to be done by both parent and child [PID=2734].
```

fork() Example

- Forking a parent process n times will result in creating $2^n - 1$ child processes.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

void main(){
    fork();
    fork();
    fork();
    printf("process\n");
}
```



fork() Example

```
#include <stdio.h>
#include <stdlib.h>
int x=0;
void main(void){
    int i;
    for(i=0;i<4;i++)
        fork();
    x=x+5;
    printf("x=%d\n",x);
}
```

```
#include <stdio.h>
#include <stdlib.h>
int x=0;
void main(void){
    int i;
    for(i=0;i<4;i++) {
        fork();
        x=x+5;
    }
    printf("x=%d\n",x);
}
```

exec() system call

- ❖ **execvp** system call used after a **fork** to replace the process' memory space with a new program.
 - Which means that the **process** having **exec system call** does **nothing except executing** the **program specified** by **exec**.
- ❖ **The parameters** passes to **execvp** are:
 1. the path of the program to execute ended with the program name as well,
 2. the second parameter is the program name,
 3. the third until the parameter before last are the arguments you need to pass to the program you are executing,
 4. and finally the last argument is **NULL**.

exec() Example

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
main(){
    pid_t pid;
    switch (pid = fork()) {
        case -1: printf("fork failed");
                break;
        case 0: execlp("/bin/ls", "ls", "-l", NULL);
                printf("exec failed");
                break;
        default: wait(NULL);
                printf("I am the parent: ls called when child is completed\n");
                exit(0);
    }
}
```

```
total 16
drwxrwxr-x 2 loay loay 4096 Oct 14 12:55 ipc
drwxrwxr-x 3 loay loay 4096 Oct 1 08:53 misc
drwxrwxr-x 2 loay loay 4096 Sep 27 18:52 processes
drwxrwxr-x 2 loay loay 4096 Oct 14 08:58 thread
I am the parent: ls called when child is completed
```

wait() System Call

❖ Usage:

```
#include <sys/types.h>
```

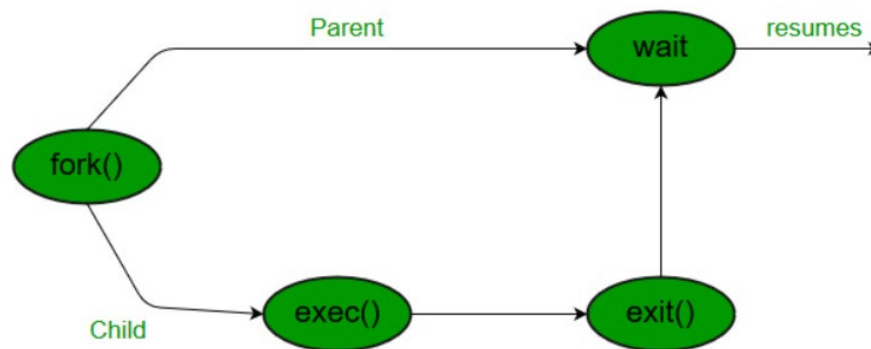
```
#include <sys/wait.h>
```

```
pid_t wait(int *status);
```

Header files to
include

Prototype

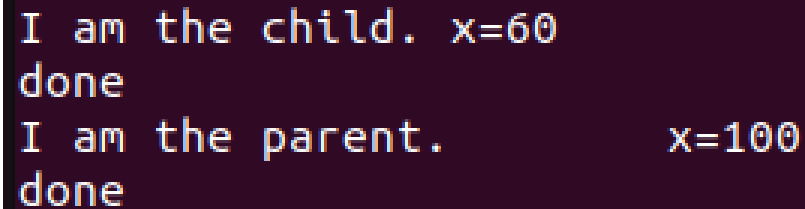
- ❖ Function: temporarily suspend the process. Often, called by a parent process to wait until its child exited. i.e. used to block until a process terminates.
- ❖ Parameters: one parameter a pointer to an integer. Useful to define exit status of child when wait returns. Or **NULL**, the parameter is ignored.
- ❖ Returned value: usually is the process id of the exiting child. If the process has no children -1 is returned.



wait() Example

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
```

```
void main(){
    int pid, y = 50, x = 50;
    pid = fork();
    switch(pid){
        case -1: printf("fork failed.");break;
        case 0: x=x+10;
                printf("I am the child.\tx=%d\n",x);break;
        default: wait(NULL);
                x=x+y;
                printf("I am the parent.\tx=%d\n",x);
    }
    printf("done\n");
}
```

A terminal window showing the output of the program. The child process prints "I am the child. x=60" and the parent process prints "I am the parent. x=100". Both processes then print "done".

```
I am the child. x=60
done
I am the parent. x=100
done
```

waitpid

❖ **pid_t waitpid (pid_t pid, int *status, int options);**

- **waitpid()** waits until a child specified by **pid** returns.
- The option **WNOHANG** is the most commonly used returning 0 until the child exits.
- **WIFEXITED(status)**: macro that checks if the returned status is set.
- **WEXITSTATUS(status)**: macro that extract the returned status to an integer value.

waitpid() Example

```
#include <stdlib.h>
#include<sys/wait.h>
#include<sys/types.h>
int main() {
    pid_t pid;
    int status, estatus;
    pid = fork();
    switch (pid) {
        case -1: printf ("fork failed\n");exit(1);
        case 0: printf ("child %d is sleeping ..... \n", getpid());
                sleep(3);
                exit(55);
        default: while (waitpid (pid, &status, WNOHANG) == 0) {
                    printf ("parent %d is waiting for child %d to exit...\n", getpid(), pid);
                    sleep (1);
                }
    }
    if (WIFEXITED (status)) {
        estatus = WEXITSTATUS (status);
        printf ("child %d has finished with exit status %d \n", pid, estatus);
    }
    return 0;
}
```

```
parent 2847 is waiting for child 2848 to exit....
child 2848 is sleeping .....
parent 2847 is waiting for child 2848 to exit....
parent 2847 is waiting for child 2848 to exit....
parent 2847 is waiting for child 2848 to exit....
child 2848 has finished with exit status 55
```

Process Termination

- ❖ Process executes last statement and asks the operating system to delete it (**exit**)
 - Output data from child to parent (via **wait**)
 - Process' resources are deallocated by operating system
- ❖ Parent may terminate execution of children processes (**abort**) for many reasons:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - If parent is exiting
 - Some operating system do not allow child to continue if its parent terminates
 - All children terminated - *cascading termination*

exit() Example

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
//multiple children of the same parent
void main() {
    int pid, i;
    for (i=0; i<10; i++) {
        pid = fork ();
        if (pid < 0)
            printf ("fork failed.");
        else if (pid > 0) {
            wait (NULL);
            printf ("Child has finished. I am the parent[PID=%d].\n", getpid());
        }
        else{
            printf ("I am the child[PID=%d].\n", getpid());
            exit (0);
        }
    }
}
```

```
I am the child[PID=3161].
Child has finished. I am the parent[PID=3160].
I am the child[PID=3162].
Child has finished. I am the parent[PID=3160].
I am the child[PID=3163].
Child has finished. I am the parent[PID=3160].
I am the child[PID=3164].
Child has finished. I am the parent[PID=3160].
I am the child[PID=3165].
Child has finished. I am the parent[PID=3160].
I am the child[PID=3166].
Child has finished. I am the parent[PID=3160].
I am the child[PID=3167].
Child has finished. I am the parent[PID=3160].
I am the child[PID=3168].
Child has finished. I am the parent[PID=3160].
I am the child[PID=3169].
Child has finished. I am the parent[PID=3160].
I am the child[PID=3170].
Child has finished. I am the parent[PID=3160].
```

Zombies and premature exits

- ❖ **Zombie**: happens if a **child exits** when its **parent is not** currently executing **wait**. The **exiting process** becomes a **zombie** occupying a **slot** in a **table** maintained in the kernel for process control. **Later** when the **parent** executes **wait**, it can **read** the **status** of the **child process** and set it to rest.

```
loay@loay-virtual-machine:~$ ./myprog &
[1] 45163
loay@loay-virtual-machine:~$ ps
  PID TTY          TIME CMD
 2642 pts/0        00:00:00 bash
 45163 pts/0        00:00:01 myprog
 45164 pts/0        00:00:01 myprog
 45165 pts/0        00:00:00 ps
loay@loay-virtual-machine:~$ kill -9 45164
loay@loay-virtual-machine:~$ ps
  PID TTY          TIME CMD
 2642 pts/0        00:00:00 bash
 45163 pts/0        00:00:26 myprog
 45164 pts/0        00:00:24 myprog <defunct>
 45169 pts/0        00:00:00 ps
loay@loay-virtual-machine:~$ kill -9 45163
loay@loay-virtual-machine:~$ ps
  PID TTY          TIME CMD
 2642 pts/0        00:00:00 bash
 45171 pts/0        00:00:00 ps
[1]+  Killed                  ./myprog
loay@loay-virtual-machine:~$
```

The diagram illustrates the process lifecycle through a series of terminal commands and the output of the 'ps' command. Annotations in blue boxes with yellow arrows point to specific processes or actions:

- parent**: Points to PID 2642 (bash) in the first 'ps' output.
- child**: Points to PID 45163 (myprog) in the first 'ps' output.
- Kill child**: Points to the command 'kill -9 45164'.
- Zombie child**: Points to PID 45164 (myprog <defunct>) in the second 'ps' output.
- Kill parent**: Points to the command 'kill -9 45163'.
- Killed**: Points to the '[1]+ Killed' message at the bottom of the terminal.

Zombies and premature exits

- ❖ **Premature exit:** happens if the **parent exits** while **one or more children** are **still running**. In this case, the **children** (including zombies) are **adopted** by the **init** process (process-id = 1).